NGO Volunteer and Donation Management System (VDMS) — Project Master Sheet

Prepared by Shubh Srivastava (2024300249) and Akshit Sharma (2024300234) | CE-302 Software Engineering Laboratory | TE CE D1 Based on: SRS Release 1.0 (30/07/2026) | Status: Ideation complete, moving to design and build

1. Project Snapshot

Item	Detail
Product	Web platform for an NGO to manage volunteers, donation campaigns, donors, events and fund utilization
Problem	Spreadsheet and register based tracking is error-prone, unauditable, and does not scale
Users	Admin (NGO staff), Volunteer, Donor (registered or guest), plus the public (read-only transparency pages)
Release 1.0 scope	Simulated payments, single NGO, single currency (INR), email receipts, role-based dashboards, reports
Stack (fixed by CO-2)	React frontend, FastAPI backend, SQLite for demo, PostgreSQL for production-style deployment
One-line pitch	A transparent, auditable NGO platform where every rupee donated can be traced to the impact it created, and every volunteer hour is recorded and recognised.

Goals

1. Replace manual tracking with one reliable system of record.
2. Make donations quick (4 steps or fewer) and trustworthy (verifiable receipts).
3. Make volunteer coordination painless (sign-up, tasks, attendance, hours).
4. Make fund usage transparent and auditable.
5. Deliver a clean, well-documented, testable system within one semester.

2. What We Have Already Thought Of (SRS Baseline)

Module	Features committed in the SRS
Volunteer management (3.1)	Register with name, email, phone, interests; unique email and phone check; Admin approval; profile edit; Admin profile view; deactivate; view assigned tasks
Campaign management (3.2)	Create with validation; Draft to Active to Closed lifecycle; publish; public active list with progress; live amount-raised update; auto-close on end date or goal; manual close; edit rules (goal locked once Active)
Donation processing (3.3)	Initiate and validate amount; guest donation; simulated payment (card, UPI, net banking); confirm screen; success and failure handling; receipt (PDF or on-screen) with unique number; receipt email; donation history
Donor management (3.4)	Auto-create donor record; Admin search by name, email, phone; donor contribution view; profile edit; duplicate-donor flag
Events and tasks (3.5)	Create and publish events; approved-volunteer sign-up with slot limit; full-event message; task assignment; attendance marking; cancel with notifications; participation history
Fund utilization (3.6)	Record spend with category, amount, reference; block overspend; per-campaign report; PDF/CSV export; Admin dashboard totals
Cross-cutting	RBAC, HTTPS, salted password hashing, audit log of fund entries, responsive UI, inline validation, Swagger docs, 100 concurrent users, 3s page load, 5s donation confirmation

3. Gaps Found in the SRS (and Proposed Fixes)

These are things the current SRS does not say, or says ambiguously. Fixing them makes the project more complete and easier to defend in evaluation. Each should be added to the SRS as a new or revised requirement (version 1.1).

#	Gap	Proposed fix
G1	No authentication requirements: no login, logout, donor registration, email verification, password reset, or session handling	Add an Auth feature (section 5, F-AUTH) with JWT access and refresh tokens, email verification, password reset
G2	Volunteer approval has no Reject outcome and no reason; status list lacks rejected	Add status rejected and a rejection reason; notify the applicant by email
G3	Donation status lacks Cancelled (Donor cancels at the confirm screen)	Add cancelled to BR-2 and the donation state diagram
G4	Campaign can be Draft, Active, Closed, but a campaign with a future start date has no state	Add Scheduled state, auto-activated by a scheduled job on the start date
G5	Goal reached auto-closes the campaign, but the last donation may overshoot the goal, and two simultaneous donations can race	Define a rule: a donation that crosses the goal is accepted, then the campaign closes. Use database transactions and row locking so totals never drift
G6	Fund utilization can be recorded against an Active campaign, but nothing stops spending money not yet raised	Already blocked by cumulative check; also add that edits and deletes of entries are not allowed, only reversal entries
G7	Admin is a single role; no distinction between a super admin and a staff member	Add roles: Super Admin (manages users, settings) and Staff Admin (day-to-day operations)
G8	Event lifecycle lacks transitions: published to completed is not defined, and volunteer self-cancellation is missing	Define the event state diagram; add Event.Signup.Cancel with a cut-off time
G9	No waitlist when an event is full	Add a waitlist with automatic promotion when a slot opens
G10	Duplicate and conflicting sign-ups not addressed	Reject duplicate sign-ups and warn on time-overlapping events
G11	Receipt rules: no format for receipt numbers, no tax-benefit information, and PAN is optional with no validation	Define the receipt number format (VDMS-YYYY-000001), validate PAN with a regex, and add an 80G-ready receipt template (TBD: confirm with NGO)

#	Gap	Proposed fix
G12	Refunds and failed-then-debited payments not covered	Add Refunded status (simulated) with an Admin action and an audit entry
G13	Privacy and data protection missing (India DPDP Act 2023): consent, data export, deletion	Add consent checkbox, privacy policy page, donor and volunteer data export, account deletion with anonymization
G14	Audit trail covers only fund utilization	Extend the audit log to campaigns, donations, approvals, role changes, and logins
G15	No notification model beyond email on two events	Add in-app notifications plus email templates for all key events
G16	Search, filter, pagination, and sorting are not specified for lists	Add them as a general requirement for all list screens
G17	No accessibility, localisation, or error-handling requirements	Add WCAG 2.1 AA basics, English plus Hindi readiness, standard error format and friendly error pages
G18	Backup, logging, and monitoring not mentioned	Add daily DB backup, structured logs, and a health-check endpoint
G19	Typo in the SRS user class heading ('Admin (NGO Staff) — favoured')	Remove the word 'favoured' before baselining
G20	Donor record merge is only flagged, never performed	Add Donor.Merge.Execute for Admin, with an audit entry

4. Core Features (Must Build — Release 1.0 MVP)

If time runs short, these alone make a complete, demonstrable product.

1. Authentication and role-based access (Admin, Volunteer, Donor).
2. Campaign lifecycle: create, publish, view, progress, close.
3. Donation flow with simulated payment, success and failure paths.
4. Receipt generation (PDF) and email.
5. Donor records and donation history.
6. Volunteer registration and Admin approval.
7. Event creation, publishing, sign-up with slot limit.
8. Task assignment and attendance marking.
9. Fund utilization recording with the overspend check.

10. Fund report per campaign with PDF/CSV export.
11. Role-based dashboards (Admin totals, Volunteer events, Donor history).
12. Audit log, validation, and documented REST API.

5. Proposed Additional Features (Fill the Gaps and Improve the Product)

Priority key: P1 = strongly recommended for Release 1.0, P2 = if time allows, P3 = future release.

F-AUTH Authentication and Account Management (P1)

- Email and password login, logout, donor self-registration.
- Email verification link and password reset link (time-limited tokens).
- Account lockout after repeated failures; show last login time.
- Guest donors can claim their donations later by registering with the same email.

F-VOL Volunteer Enhancements

- Reject with reason, plus re-apply (P1).
- Skills, languages, availability slots, and city on the profile (P1).
- Self-cancel an event sign-up before a cut-off (P1).
- Waitlist with auto-promotion (P2).
- Volunteer hours tracking, derived from attendance (P1).
- Downloadable participation certificate with QR verification (P2).
- Badges and milestones, for example 5 events or 25 hours (P3).

F-CAMP Campaign Enhancements

- Scheduled state with auto-activation (P1).
- Cover image, gallery, story, and beneficiary details (P2).
- Milestones (for example 25%, 50%, 100%) with automatic progress updates (P2).
- Campaign updates feed where Admin posts progress and photos (P2).
- Suggested donation amounts with an impact label, such as Rs 500 feeds 10 people (P2).
- Shareable campaign link and QR code (P2).
- Clone a past campaign (P3).

F-DON Donation Enhancements

- Cancelled and Refunded statuses (P1).
- Idempotency key so a double-click or retry never creates two donations (P1).
- Dedicate a donation to a person, or donate anonymously (P2).
- Optional donor message and public donor wall, respecting anonymity (P2).
- Simulated recurring donations (monthly) using the scheduler, to demonstrate the TBD-2 idea (P3).
- Receipt with 80G fields and QR verification link (P1).

F-EVENT Event Enhancements

- Event categories and filters; event reminders by email 24 hours before (P2).
- Waitlist, time-conflict warning (P2).
- QR check-in: each volunteer has a personal QR, Admin scans to mark attendance (P2, also a USP).
- Post-event summary: volunteers attended, hours, photos (P3).
- Link an event to a campaign so impact rolls up to it (P2).

F-FUND Fund Utilization Enhancements

- Attach invoice scan (PDF or image) to a utilization entry (P2).
- Reversal entry instead of edit or delete, keeping history intact (P1).
- Charts: raised vs utilized, category-wise pie, monthly trend (P1).
- Approval workflow: Staff proposes, Super Admin approves large spends above a threshold (P3).

F-NOTIF Notifications and Communication

- In-app notification bell plus email templates for: approval or rejection, event sign-up, reminder, cancellation, donation receipt, campaign closed, milestone reached (P1 for email, P2 for in-app).
- Admin broadcast message to all volunteers or an event's volunteers (P2).
- SMS or WhatsApp (P3, future release as in the SRS).

F-ANALYTICS Analytics and Reporting

- Admin dashboard: total raised, utilized, balance, active campaigns, new donors, active volunteers, upcoming events (P1).
- Donor analytics: repeat donor rate, average donation, top campaigns (P2).

- Volunteer analytics: total hours, attendance rate, no-show rate (P2).
- Exportable annual report (PDF) for the NGO (P3).

F-ADMIN Administration and Governance

- Super Admin and Staff Admin roles (P1).
- Full audit log viewer with filters (P1).
- System settings: max donation limit (resolves TBD-1), cancel cut-off, receipt prefix (P1).
- Donor merge execution (P2).
- Soft delete everywhere, so no history is lost (P1).

F-PUBLIC Public and Trust Features

- Public landing page: mission, live counters, featured campaigns (P1).
- Public Transparency page (see USP 1) (P1).
- Receipt verification page that checks a receipt number (P1).
- FAQ and help centre, privacy policy, terms (P1, covers section 2.6 of the SRS).

F-QUALITY Platform Quality

- Search, filter, sort, and pagination on every list (P1).
- Accessibility basics and keyboard navigation (P2); Hindi-ready i18n structure (P3).
- Dark mode (P3).
- Standard error response format, rate limiting on login and donation endpoints (P1).

6. USP Features (What Makes This Project Stand Out)

1. **Public Transparency Ledger.** Anyone can open a campaign and see raised, utilized, remaining, and each spend category with its supporting reference. Invoice numbers and (optionally) redacted invoice images are shown. Most small NGOs cannot offer this; it builds donor trust directly from the data we already capture.
2. **Verifiable Receipts.** Every receipt carries a unique number, a QR code, and a hash of its contents. A public verify page confirms authenticity. This stops forged receipts and reflects a real 80G workflow.
3. **Tamper-Evident Audit Trail.** Each audit entry stores a hash of the previous entry (hash chain). A single Admin screen can verify the whole chain. Strong talking point for security and auditability.
4. **Smart Volunteer Matching.** For each event, the system ranks approved volunteers using a transparent score: interest match, skills, availability, past attendance rate, and distance in city. The Admin sees why each person is suggested. Rule-based, no black-

box ML needed, but extensible.

5. **QR Check-in with Auto Hours and Certificates.** Volunteers show a QR at the event; scanning marks attendance, adds hours to their profile, and later issues a verifiable certificate. Removes the manual attendance sheet.
6. **Impact-First Campaign Pages.** Campaigns show milestones, impact-per-amount labels, and an updates feed, so donors see outcomes, not just a progress bar.
7. **One Person, Two Roles.** A single account can be both Donor and Volunteer. The profile shows combined contribution (money plus hours), enabling recognition and retention.

7. Technical Architecture

7.1 Style

Three-tier web application with a decoupled SPA frontend and a stateless REST API (modular monolith). A modular monolith is right for a two-person semester project: simple to deploy, easy to test, and clean module boundaries allow future splitting.

7.2 Layered View

Layer	Responsibility	Technology
Presentation	SPA, routing, forms, charts, role-based UI	React, TypeScript, Vite
API	HTTP routes, request and response validation, auth dependencies	FastAPI, Pydantic v2
Service (business logic)	Rules such as campaign lifecycle, overspend check, slot allocation, matching	Plain Python service classes
Data access	Repositories, transactions, migrations	SQLAlchemy 2.0, Alembic
Database	Persistent storage	SQLite (demo), PostgreSQL (deployment)
Background jobs	Auto-close campaigns, activate scheduled campaigns, event reminders, email queue	APScheduler (or FastAPI BackgroundTasks for small jobs)
Integrations	Email, payment (mock now, real later), PDF, QR	SMTP, PaymentProvider interface, ReportLab, qrcode

7.3 Key Design Decisions

- **Payment abstraction.** Define a PaymentProvider interface with authorize and capture methods. Release 1.0 ships MockPaymentProvider (configurable success rate, delay, forced failure for demos). Later, RazorpayProvider can replace it with no change to the donation service (satisfies SI-2.2).
- **Money as integers.** Store amounts in paise as integers (or Decimal with 2 places) to meet CO-3.
- **Transactions and locking.** The donation completion step updates the donation, the campaign total, and the audit entry in one transaction, with a row lock on the campaign, so concurrent donations stay consistent.
- **State machines in code.** Campaign, donation, event, and volunteer statuses are enforced by explicit allowed-transition tables in the service layer, mirroring the state diagrams in Appendix B.
- **Stateless auth.** JWT access tokens (15 minutes) plus refresh tokens (7 days), role claim included, checked by a FastAPI dependency.

- **Soft delete and audit.** Records are never physically removed; every sensitive change writes an audit row.
- **Config via environment.** Secrets and settings in a .env file, never in the repository.

7.4 Request Flow Example (Donation)

1. Donor opens a campaign, enters amount, picks payment method, confirms.
2. Frontend calls POST /donations with an idempotency key.
3. Donation service validates amount and campaign state, creates a Pending donation.
4. PaymentProvider simulates authorization; result returned.
5. On success (one transaction): mark Completed, add to campaign total, generate receipt number, write audit row, check goal and close if reached.
6. Background task generates the PDF and sends the email.
7. API returns the result; UI shows the receipt and a download link. On failure: status Failed, no total change, retry offered.

7.5 Suggested Repository Structure

```
vdms/
  backend/
    app/
      main.py, config.py, database.py
      core/        (security, deps, errors, logging)
      models/      (SQLAlchemy models)
      schemas/     (Pydantic schemas)
      repositories/
      services/    (auth, campaign, donation, volunteer, event, fund, report,
notification, audit)
      api/v1/      (routers per module)
      integrations/ (payment, email, pdf, qr)
      jobs/        (scheduler tasks)
    alembic/, tests/
  frontend/
    src/
      app/ (router, providers)
      features/ (auth, campaigns, donations, volunteers, events, funds,
admin)
      components/ (ui, layout, charts)
      lib/ (api client, hooks, utils), pages/
  docs/ (SRS, UML, DFD, API, user guide)
  docker-compose.yml, .github/workflows/
```

8. Technology Stack

Area	Choice	Why
Frontend framework	React 18 with TypeScript, Vite	Required by CO-2; type safety; fast dev builds
Styling and UI	Tailwind CSS with shadcn/ui components	Consistent, responsive, accessible components quickly
Routing and state	React Router; TanStack Query for server state; Zustand for small client state	Less boilerplate than Redux
Forms and validation	React Hook Form with Zod	Meets UI-4 inline validation
Charts	Recharts	Simple dashboards
Backend	FastAPI on Python 3.11+, Uvicorn	Required by CO-2; automatic OpenAPI satisfies CO-1
ORM and migrations	SQLAlchemy 2.0, Alembic	Clean models; versioned schema
Validation	Pydantic v2	Typed request and response models
Auth	PyJWT, Argon2 or bcrypt (via passlib)	Meets SE-3 salted hashing
Database	SQLite (demo), PostgreSQL (deployment)	Matches OE-3
Jobs	APScheduler	Auto-close, reminders, scheduled activation
Email	fastapi-mail or smtplib; Mailtrap or Gmail SMTP for testing	Meets DE-1
PDF and QR	ReportLab (or WeasyPrint), qrcode	Receipts, reports, certificates
Testing	pytest, httpx; Vitest, React Testing Library; Playwright for end-to-end	Covers all layers
Quality	Ruff, Black, ESLint, Prettier, pre-commit	Consistent code

Area	Choice	Why
DevOps	Docker, docker-compose, GitHub Actions	Reproducible runs and CI
Hosting (demo)	Frontend on Vercel or Netlify; backend and PostgreSQL on Render or Railway	Free tiers, HTTPS by default (CI-3)
Docs and design	Swagger UI, draw.io or PlantUML for UML and DFD, Figma for wireframes	Lab deliverables

9. Frontend Plan

9.1 Screens by Role

Role	Screens
Public	Landing, campaign list and detail, transparency view, receipt verification, event list (read-only), FAQ, login, register, forgot password
Donor	Donate flow (amount, details, payment, confirm, result), dashboard, donation history with receipt download, profile
Volunteer	Dashboard, browse and sign up for events, my events and tasks, hours and certificates, profile, notifications
Admin	Dashboard with charts, volunteer approvals, volunteer directory, campaign manager (create, edit, publish, close), event manager, attendance and QR scanner, donor directory, fund utilization entry and reports, audit log, settings (Super Admin)

9.2 Frontend Points

- Mobile-first responsive layouts (UI-3); test at 360px width.
- Route guards by role; redirect unauthenticated users to login; hide menu items the role cannot use.
- Donation flow is a 4-step wizard with a progress indicator (Usability-1).

- Central API client with token refresh, error normalisation, and request cancellation.
- Loading skeletons, empty states, and friendly error screens for every list.
- Debounced search, server-side pagination, and URL-synced filters.
- Toast notifications for success and failure; confirmation dialogs for destructive actions.
- Accessibility: labels, focus states, colour contrast, keyboard-friendly forms.
- Theme tokens (colours, spacing) defined once so the UI looks consistent.
- Charts: raised vs goal bar, utilization pie, monthly donation line.

10. Backend Plan

10.1 API Surface (versioned under /api/v1)

Module	Key endpoints
Auth	POST /auth/register, /auth/login, /auth/refresh, /auth/logout, /auth/verify-email, /auth/forgot-password, /auth/reset-password, GET /auth/me
Volunteers	POST /volunteers (apply), GET /volunteers, GET /volunteers/{id}, PATCH /volunteers/{id}, POST /volunteers/{id}/approve, /reject, /deactivate, GET /volunteers/me/events, /me/hours
Campaigns	POST /campaigns, GET /campaigns (public, filters), GET /campaigns/{id}, PATCH /campaigns/{id}, POST /campaigns/{id}/publish, /close, GET /campaigns/{id}/updates, POST /campaigns/{id}/updates
Donations	POST /donations, GET /donations/{id}, POST /donations/{id}/retry, /cancel, /refund, GET /donations/me, GET /donations/{id}/receipt, GET /receipts/verify/{number}
Donors	GET /donors, GET /donors/{id}, PATCH /donors/me, GET /donors/duplicates, POST /donors/merge
Events	POST /events, GET /events, GET /events/{id}, PATCH /events/{id}, POST /events/{id}/publish, /cancel, /signup, DELETE /events/{id}/signup, POST /events/{id}/registrations/{rid}/task, /attendance, /checkin, GET /events/{id}/suggested-volunteers

Module	Key endpoints
Funds	POST /campaigns/{id}/utilizations, GET /campaigns/{id}/utilizations, POST /utilizations/{id}/reverse, GET /campaigns/{id}/fund-report, GET /campaigns/{id}/fund-report/export?format=pdf or csv, GET /dashboard/summary
Admin	GET /audit-logs, GET /audit-logs/verify, GET and PUT /settings, GET /notifications, POST /notifications/{id}/read
System	GET /health, GET /docs (Swagger)

10.2 Backend Points

- Dependency-injected auth: get_current_user and require_role(Admin) used on every protected route (SE-4).
- Business rules live in services, not routers, so they are unit-testable.
- Standard error shape: code, message, field errors, request id.
- Idempotency keys stored for donation creation.
- Rate limiting on login, register, and donation endpoints.
- CORS restricted to the frontend origin; security headers added.
- Structured JSON logging with a request id; never log passwords or payment details (SE-3).
- Scheduler jobs: close expired campaigns (hourly), activate scheduled campaigns, send event reminders, purge expired tokens.
- Seed script to generate demo data (campaigns, donors, volunteers, events) for evaluation.
- Pagination, sorting, and filtering handled by a shared helper.

11. Data Model (Extends Appendix A)

New tables: User (login identity: id, email, password_hash, role, is_verified, status), AuditLog (id, actor_id, action, entity, entity_id, old and new values, prev_hash, hash, timestamp), Notification, EmailLog, CampaignUpdate, CampaignMilestone, Receipt (number, donation_id, hash, issued_at), Setting (key, value), RefreshToken or PasswordResetToken, IdempotencyKey, VolunteerSkill, EventWaitlist, Certificate.

Changes to existing entities:

- Volunteer: add user_id, city, skills, availability, total_hours; status adds rejected and reject_reason.
- Donor: add user_id (nullable for guests), pan (validated), consent_at, is_anonymous_default.
- Campaign: status adds scheduled; add cover_image, impact_note, created_by, version.
- Donation: status adds cancelled and refunded; add idempotency_key, is_anonymous, dedication, message, failure_reason.
- Event: add campaign_id (nullable), category, cancel_cutoff, check-in token.
- EventRegistration: add checked_in_at, hours_credited, status adds waitlisted and cancelled.
- FundUtilization: add invoice_file, reversed_of (self reference), status.

Key relationships: User 1-1 Volunteer or Donor or Admin profile; Donor 1-m Donation m-1 Campaign; Campaign 1-m FundUtilization; Volunteer m-m Event through EventRegistration; Campaign 1-m Event (optional).

Integrity rules: unique email and phone; CHECK amount greater than 0; sum of utilizations not above amount raised (service plus DB-level check or trigger on PostgreSQL); indexes on campaign status and end date, donation donor and campaign, event date.

12. Security, Privacy and Compliance

- HTTPS everywhere (CI-3, SE-1); HSTS in deployment.
- Argon2 or bcrypt hashing (SE-3); password strength rules; lockout after repeated failures.
- JWT with short expiry, refresh rotation, and logout revocation.
- RBAC enforced on the server, never only in the UI (SE-4); object-level checks so donors see only their own history (SE-5).
- Input validation (Pydantic and Zod), parameterised queries through the ORM, output encoding, file upload type and size limits.
- CSRF protection if cookies are used; otherwise store tokens carefully (httpOnly cookie recommended).
- No card data stored; mock payments only (CO-4). PAN is masked in the UI and only shown in full to Admin.
- DPDP Act readiness: consent capture, privacy policy, data export, deletion by anonymization.
- Audit log with hash chain (SE-6 extended); dependency scanning in CI.

13. Non-Functional Targets (Refined)

Area	Target	How we verify
Performance	Listing pages under 3s, donation confirmed under 5s, 100 concurrent users	Locust or k6 load test script
Availability	95% during evaluation	Health check and uptime monitor
Usability	Donation in 4 steps or fewer	Usability test with 5 first-time users
Maintainability	Documented API, 70%+ backend test coverage	Swagger, coverage report in CI
Portability	Python 3.11+, Node 18+, any OS	Docker image runs on Windows and Linux
Accessibility	Lighthouse accessibility 90+	Lighthouse report

14. Testing Strategy

- **Unit tests:** services (campaign transitions, overspend check, slot allocation, receipt numbering, matching score).
- **API tests:** every endpoint with success, validation failure, and permission failure cases.
- **Concurrency tests:** two simultaneous donations near the goal; two volunteers taking the last slot.
- **Frontend tests:** form validation, route guards, donation wizard.
- **End-to-end (Playwright):** donate as guest, volunteer apply then approve then sign up then attend, fund spend then report.
- **Non-functional:** load test, Lighthouse, basic security scan (OWASP ZAP baseline).
- **Traceability:** a table mapping each SRS requirement ID to its test case (useful for evaluation).

15. DevOps and Deployment

- Git with feature branches and pull requests; main always deployable; conventional commit messages.
- GitHub Actions: lint, test, build on every push; deploy on merge to main.

- Docker images for backend and frontend; docker-compose for local run with PostgreSQL.
- Environments: local (SQLite), staging and demo (PostgreSQL on Render or Railway).
- Alembic migrations run on deploy; daily database backup script.
- Seeded demo accounts: one Super Admin, one Staff Admin, three Volunteers, three Donors.

16. Delivery Roadmap

Phase	Focus	Deliverables
1. Design	Finalise SRS v1.1, UML (use case, class, sequence, activity), DFD, wireframes, database schema	Design documents, Figma wireframes, ERD
2. Foundation	Repo, CI, auth, RBAC, base UI shell, database and migrations, audit log	Login and role dashboards working
3. Money flow	Campaigns, donation flow with mock payment, receipts, donor records	Core features 2 to 5 complete
4. People flow	Volunteers, approvals, events, sign-ups, tasks, attendance	Core features 6 to 8 complete
5. Funds and reports	Fund utilization, reports, exports, dashboards, charts	Core features 9 to 11 complete
6. USP and polish	Transparency page, verifiable receipts, QR check-in, matching, notifications	USP 1, 2, 5 first; others as time allows
7. Hardening	Testing, load test, security review, accessibility, docs, user guide, demo data	Release 1.0 candidate

Suggested split: Shubh takes backend core, payments, funds, audit and security; Akshit takes frontend, volunteers and events, notifications and QR; both own testing and documentation of their own modules and cross-review each other's pull requests.

17. Mapping to Remaining Lab Experiments

Experiment	Artifact produced from this master sheet
Use case modelling	Actors: Admin, Volunteer, Donor, Guest Donor, Email Service, Payment Provider. Major use cases: Donate, Register as Volunteer, Approve Volunteer, Manage Campaign, Manage Event, Record Fund Utilization, View Report
Class diagram	Entities from section 11 with services as controller classes
Sequence diagrams	Donation flow, volunteer sign-up with waitlist, campaign auto-close
Activity diagrams	Volunteer onboarding, campaign lifecycle, fund spend validation
DFD (levels 0 to 2)	Level 0 context from SRS Figure 1; Level 1 with processes: Auth, Campaigns, Donations, Volunteers and Events, Funds and Reports
State diagrams	Campaign, donation, event, volunteer (updated per gaps G2 to G4, G8)

18. Risks and Mitigations

Risk	Impact	Mitigation
Scope too large for one semester	Incomplete core	Build strictly in core-then-P1-then-P2 order; USPs 1 and 2 are cheap and give the most value
Concurrency bugs in totals and slots	Wrong money figures	Transactions, row locks, and dedicated tests
Email service unavailable in demo	Receipts not delivered	Email log table, console and Mailtrap fallback, on-screen receipt always available
Team of two, uneven progress	Delays	Weekly sync, module ownership, small pull requests
Deployment issues on demo day	Failed demo	Docker, a recorded backup demo video, and local fallback

Risk	Impact	Mitigation
Privacy mistakes with real data	Compliance issue	Use only synthetic data; no real PAN or card numbers

19. Resolutions for TBD Items

TBD	Proposed resolution
TBD-1 Maximum donation limit	Make it a Super Admin setting; default Rs 1,00,000 per donation
TBD-2 Recurring donations	Out of scope for 1.0; optional simulated monthly donation as a P3 demo
TBD-3 Payment gateway	Razorpay (INR, UPI, good test mode and documentation)
TBD-4 Multi-branch	Out of scope; keep a nullable branch_id design note for future

20. Demo Script (Evaluation Day)

1. Open the public landing page and a campaign with live progress and the transparency view.
2. Donate as a guest: 4 steps, simulated success, receipt PDF, verify it on the public verify page.
3. Force a payment failure, show the retry path and that the total did not change.
4. Register a volunteer, reject then approve as Admin, show the email.
5. Admin creates and publishes an event; volunteer signs up; a second volunteer hits the full or waitlist case.
6. Admin checks in a volunteer using the QR; hours update on the volunteer dashboard.
7. Admin records a fund utilization, then tries to overspend and sees the block.
8. Show the fund report, export to PDF, and the audit log hash verification.
9. Show Swagger docs, test coverage, and the CI pipeline.

21. Future Scope (Beyond Release 1.0)

Real payment gateway integration, recurring donations, SMS and WhatsApp notifications, native mobile app, multi-NGO and multi-branch support, accounting software integration (for example Tally), embeddable donation widget, Hindi and regional language support, and ML-assisted volunteer recommendation.

22. Immediate Next Steps

1. Decide which P1 and P2 items you want in Release 1.0 and update the SRS to version 1.1 (fix gaps G1 to G20 selected).
2. Lock the data model (section 11) and draw the ERD.
3. Start the use case and DFD experiments using section 17 as the input.
4. Create the repository, CI, and folder structure from section 7.5.
5. Build the foundation phase (auth, RBAC, audit log) before any feature work.